

DD_SIP-02 - Package Launch Lifecycle - Developer Implementation Guide

Version: v1.0 Bundle: 09_catalogs Companion DD: DD_SIP-02-Package_Launch_Lifecycle-v1.0

1. Implementation Goal

Build the package lifecycle service that turns SIP-01 package definitions into sellable packages.

SIP-01 creates package data. SIP-02 decides when that data is ready, approved, active, available by channel/franchise/region, suspended, or retired.

Main implementation flow:

```
sequenceDiagram
    participant UI as Product Backoffice
    participant SIP2 as SIP-02
    participant SIP1 as SIP-01
    participant PLM as PLM/BIL config APIs
    participant RUL as Drools/KIE
    participant APR as EM-CFG-04
    participant WKR as Catalog Worker

    UI->>SIP2: POST /api/package-launch-plans
    SIP2->>SIP1: load package and version
    UI->>SIP2: POST /validate
    SIP2->>PLM: validate service/tax/wallet/billable events
    SIP2->>RUL: launch readiness rules
    UI->>SIP2: POST /submit-review
    SIP2->>APR: evaluate/start approval if required
    APR-->>SIP2: approval callback
    WKR->>SIP2: activate when due
    SIP2->>SIP1: update package current version/status
    SIP2-->>UI: package is sellable through available API
```

2. Step 1 - Create Tables

Create the SIP-02 tables:

1. package_launch_plan
2. package_launch_check
3. package_availability
4. package_version_cutover
5. package_lifecycle_event
6. package_retirement_plan

Implementation note:

- Store SIP-02 tables in the catalog schema.
- Use SIP-01 package and package_version as referenced domain records.
- Do not duplicate package price/composition into SIP-02 except snapshots needed for audit or events.

Repository methods:

```
interface PackageLaunchPlanRepository {
    PackageLaunchPlan save(PackageLaunchPlan plan);
    Optional<PackageLaunchPlan> findById(String launchPlanId);
    List<PackageLaunchPlan> findDueForActivation(Instant now, int limit);
    Optional<PackageLaunchPlan> findByApprovalRequestId(String approvalRequestId);
}

interface PackageAvailabilityRepository {
    List<PackageAvailability> findAvailable(
        String operatorCode,
        String channelCode,
        String franchiseId,
```

```

        String techRegionCode,
        Instant onDate
    );
}

```

3. Step 2 - Create Launch Plan Endpoint

Request:

```

POST /api/package-launch-plans
Idempotency-Key: package-launch-FIBER_100M_RES-pkv-202607
Content-Type: application/json

```

```

{
  "operatorCode": "WIK",
  "packageId": "pkg_01HX_100M",
  "packageCode": "FIBER_100M_RES",
  "packageVersionId": "pkv-100m-202607",
  "launchType": "FIRST_LAUNCH",
  "requestedLaunchAt": "2026-07-01T00:00:00+03:00",
  "effectiveTimezone": "Africa/Nairobi",
  "availability": [
    {
      "franchiseId": "fr-019",
      "techRegionCode": "NRB-WEST",
      "channelCode": "SALES_APP"
    }
  ]
}

```

On reception on catalog API pod:

1. Validate required fields.
2. Check idempotency key.
3. Load package and package version through SIP-01 service/repository.
4. Confirm package version belongs to package.
5. Create package_launch_plan in DRAFT.
6. Store requested availability as draft metadata or create package_availability rows in SCHEDULED only after approval. Prefer creating rows after approval to avoid exposing unapproved availability.
7. Write package_lifecycle_event with PACKAGE_LAUNCH_PLAN_CREATED.
8. Publish outbox event PackageLaunchPlanCreated.

Response:

```

{
  "launchPlanId": "plp-001",
  "status": "DRAFT"
}

```

4. Step 3 - Validate Launch Plan

Request:

```

POST /api/package-launch-plans/plp-001/validate

```

Validation flow:

1. Load launch plan.
2. Load SIP-01 package.
3. Load SIP-01 package version.
4. Load package composition services from PLM-CFG-01.
5. Validate service statuses are active.
6. Validate tax group through PLM-CFG-02.
7. Validate wallet binding through PLM-CFG-03.

8. Validate billing event compatibility through BIL-CFG-01.
9. Validate availability scope is inside package target franchise/region arrays.
10. Build Drools launch readiness facts.
11. Run package-launch-readiness.drl.
12. Store one package_launch_check row per check.
13. If no FAIL, move plan to READY_FOR_REVIEW.

Example check rows:

```
[
  {
    "checkCode": "PACKAGE_VERSION_EXISTS",
    "checkStatus": "PASS",
    "sourceModuleCode": "SIP-01",
    "message": "Package version exists and belongs to package."
  },
  {
    "checkCode": "SERVICE_ACTIVE",
    "checkStatus": "PASS",
    "sourceModuleCode": "PLM-CFG-01",
    "message": "All services are active."
  },
  {
    "checkCode": "SALES_APP_REGION_ALLOWED",
    "checkStatus": "WARN",
    "sourceModuleCode": "DROOLS",
    "message": "Sales app launch is allowed but requires commercial approval."
  }
]
```

If any check is FAIL, keep plan in DRAFT or VALIDATION_FAILED if you implement that internal state. The public state can remain DRAFT with failed checks visible.

5. Step 4 - Drools Facts For Launch Readiness

Create facts:

```
public record PackageLaunchFact(
    String operatorCode,
    String packageId,
    String packageCode,
    String packageVersionId,
    String launchType,
    Instant requestedLaunchAt,
    List<String> channelCodes,
    List<String> franchiseIds,
    List<String> techRegionCodes,
    BigDecimal price,
    String currency,
    int serviceCount
) {}

public class PackageLaunchDecision {
    private final List<LaunchCheckResult> checks = new ArrayList<>();
    private boolean approvalRequired;
    private String approvalPolicyCode;

    public void fail(String checkCode, String message) { ... }
    public void warn(String checkCode, String message) { ... }
    public void pass(String checkCode, String message) { ... }
}
```

6. Step 5 - Drools Rule Samples

6.1 Sales App First Launch Requires Approval

```
package com.sophix.catalog.package_lifecycle

rule "First launch to sales app requires commercial approval"
when
    $fact : PackageLaunchFact(launchType == "FIRST_LAUNCH", channelCodes contains "SALES_APP")
```

```

    $decision : PackageLaunchDecision()
  then
    $decision.warn("SALES_APP_FIRST_LAUNCH_APPROVAL", "First sales app launch requires approval.");
    $decision.setApprovalRequired(true);
    $decision.setApprovalPolicyCode("PACKAGE_LAUNCH_COMMERCIAL");
  end
end

```

6.2 Price Above Threshold Requires Finance Approval

```

rule "High price package requires finance approval"
when
  $fact : PackageLaunchFact(currency == "KES", price >= 10000.00)
  $decision : PackageLaunchDecision()
then
  $decision.setApprovalRequired(true);
  $decision.setApprovalPolicyCode("PACKAGE_LAUNCH_FINANCE");
  $decision.warn("HIGH_PRICE_FINANCE_APPROVAL", "Package price is above finance approval threshold.");
end

```

6.3 Block Empty Service Composition

```

rule "Package must contain at least one service"
when
  $fact : PackageLaunchFact(serviceCount == 0)
  $decision : PackageLaunchDecision()
then
  $decision.fail("PACKAGE_HAS_NO_SERVICE", "Package cannot launch without service composition.");
end

```

6.4 Region Expansion Requires Technical Readiness

```

rule "Region expansion requires technical readiness check"
when
  $fact : PackageLaunchFact(launchType == "REGION_EXPANSION")
  $decision : PackageLaunchDecision()
then
  $decision.warn("REGION_EXPANSION_TECH_CHECK", "Confirm technical readiness for expanded region.");
end

```

The first version can keep these rules simple. The value is that future operators can change policy by configuration/rules instead of code.

7. Step 6 - Submit Review And Approval

Request:

```

POST /api/package-launch-plans/plp-001/submit-review
Content-Type: application/json

{
  "requesterUserId": "usr-product-001",
  "comment": "Ready for Nairobi launch."
}

```

SIP-02 flow:

1. Load launch plan.
2. Confirm latest validation has no FAIL.
3. Build approval context from launch plan, checks, channels, franchises, regions, price, and launch type.
4. Call EM-CFG-04 evaluation:

```

POST /api/approvals/evaluate
Content-Type: application/json

{
  "operatorCode": "WIK",
  "moduleCode": "SIP-02",
  "actionCode": "PACKAGE_LAUNCH_APPROVAL",
  "subjectType": "PACKAGE_LAUNCH_PLAN",
  "subjectId": "plp-001",
  "requesterUserId": "usr-product-001",
  "context": {
    "packageCode": "FIBER_100M_RES",
    "packageVersionId": "pkv-100m-202607",
    "launchType": "FIRST_LAUNCH",
    "requestedLaunchAt": "2026-07-01T00:00:00+03:00",

```

```

    "channelCodes": ["SALES_APP"],
    "franchiseCount": 1,
    "techRegionCount": 1,
    "price": 3000.00,
    "currency": "KES"
  }
}

```

5. If `approvalRequired = false`, mark plan APPROVED or SCHEDULED.

6. If `approvalRequired = true`, call:

```

POST /api/approvals/requests
Idempotency-Key: approval-package-launch-plp-001
Content-Type: application/json

{
  "operatorCode": "WIK",
  "policyId": "apol-package-launch-wik-v1",
  "moduleCode": "SIP-02",
  "subjectType": "PACKAGE_LAUNCH_PLAN",
  "subjectId": "plp-001",
  "requesterUserId": "usr-product-001",
  "callbackUrl": "/internal/package-launch-plans/plp-001/approval-outcome",
  "context": {
    "packageCode": "FIBER_100M_RES",
    "launchType": "FIRST_LAUNCH",
    "requestedLaunchAt": "2026-07-01T00:00:00+03:00"
  }
}

```

7. Store `approval_request_id`, `approval_mode`, and `camunda_process_instance_id` if returned.

8. Move plan to PENDING_APPROVAL.

Camunda note: SIP-02 does not start Camunda directly. EM-CFG-04 starts Camunda only when policy mode is BPMN.

8. Step 7 - Approval Callback

Endpoint:

```

POST /internal/package-launch-plans/plp-001/approval-outcome
Content-Type: application/json

{
  "approvalRequestId": "apr-plp-001",
  "outcome": "APPROVED",
  "decisionComment": "Approved for WIK Nairobi launch."
}

```

SIP-02 flow:

1. Find launch plan by id.
2. Verify `approvalRequestId` matches stored value.
3. If callback already processed, return 200 with current state.
4. If APPROVED, move to:
 - SCHEDULED if `requestedLaunchAt` is future;
 - APPROVED if activation needs manual trigger;
 - or directly activate if policy allows immediate activation and time is due.
5. If REJECTED, move to REJECTED.
6. Write `package_lifecycle_event`.
7. Publish `PackageLaunchApproved` or rejection event.

Callbacks must be idempotent because EM-CFG-04 may retry delivery.

9. Step 8 - Activation Worker

Worker job: `package-launch-activation`.

Run every minute or consume a due-queue event.

Flow:

1. Find launch plans where status = SCHEDULED and requested_launch_at <= now.
2. Lock one plan.
3. Run final validation again.
4. If any FAIL, mark activation failed/blocked and publish alert event.
5. Start transaction.
6. Update SIP-01 package master status to ACTIVE if first launch.
7. Update SIP-01 package version status to ACTIVE.
8. If version cutover, set previous active version SUPERSEDED.
9. Set package.current_version_id = package_version_id.
10. Create package_availability rows with ACTIVE.
11. Create package_version_cutover row if current version changed.
12. Create package_lifecycle_event with PACKAGE_ACTIVATED.
13. Commit.
14. Publish PackageActivated and PackageAvailabilityChanged.
15. Invalidate package availability cache.

Pseudo-code:

```
public void activateDueLaunches() {
    List<PackageLaunchPlan> plans = launchRepo.findDueForActivation(clock.now(), 50);
    for (PackageLaunchPlan plan : plans) {
        tx.required(() -> {
            PackageLaunchPlan locked = launchRepo.lock(plan.id());
            ValidationResult finalCheck = validator.validate(locked);
            if (finalCheck.hasFail()) {
                locked.markActivationBlocked(finalCheck.summary());
                lifecycleEventService.recordBlocked(locked, finalCheck);
                return;
            }

            sip01Service.activateVersion(
                locked.packageId(),
                locked.packageVersionId(),
                locked.requestedLaunchAt()
            );

            availabilityService.activateAvailability(locked);
            cutoverService.recordCutover(locked);
            locked.markActivated();
            lifecycleEventService.recordActivated(locked);
            outbox.publish(PackageActivated.from(locked));
        });
    }
}
```

10. Step 9 - Available Packages API

Endpoint:

GET /api/packages/available?operatorCode=WIK&channelCode=SALES_APP&franchiseId=fr-019&techRegionCode=NRB-WEST&onDate=2026-

Flow:

1. Validate query.
2. Check Redis/read-model cache.
3. Query package_availability where:
 - operator matches;

- status is ACTIVE;
 - channel matches;
 - franchise matches or row is global;
 - tech region matches or row is global;
 - date is between available_from and available_until.
4. Join/load SIP-01 package and package version.
 5. Exclude package if SIP-01 master status is not ACTIVE.
 6. Run optional package-availability.drl for operator-specific filters.
 7. Return package summary.

Response:

```
{
  "packages": [
    {
      "packageId": "pkg_01HX_100M",
      "packageCode": "FIBER_100M_RES",
      "packageVersionId": "pkv-100m-202607",
      "displayName": "Fiber 100M Residential",
      "price": 3000.00,
      "currency": "KES",
      "billingFrequencyDays": 30,
      "availableChannels": ["SALES_APP", "BACKOFFICE"]
    }
  ]
}
```

This API is called by:

- FUL-02 order capture;
- Backoffice order screens;
- sales outdoor app quote flow;
- self-care package change screens;
- SIP-04 bundle launch validation.

11. Step 10 - Suspend And Resume Availability

Suspend request:

```
POST /api/packages/pkg_01HX_100M/availability/suspend
Content-Type: application/json

{
  "packageVersionId": "pkv-100m-202607",
  "franchiseId": "fr-019",
  "techRegionCode": "NRB-WEST",
  "channelCode": "SALES_APP",
  "reasonCode": "TEMPORARY_PROVISIONING_ISSUE",
  "comment": "ONT stock shortage in Nairobi West."
}
```

Flow:

1. Load matching active availability rows.
2. Check permission through EM-CFG-03/RBAC.
3. If suspension policy requires approval, call EM-CFG-04 using action PACKAGE_AVAILABILITY_SUSPEND.
4. If no approval required, set availability status SUSPENDED.
5. Write lifecycle event.
6. Publish PackageAvailabilityChanged.
7. Invalidate cache.

Resume is the same pattern with action PACKAGE_AVAILABILITY_RESUME.

For simple operational toggles, approval may be disabled by EM-CFG-04 policy. For nationwide or high-impact suspension, configure approval.

12. Step 11 - Retirement Flow

Request:

```
POST /api/package-retirement-plans
Idempotency-Key: package-retire-OLD50-20260901
Content-Type: application/json

{
  "operatorCode": "WIK",
  "packageId": "pkg_01HX_OLDS0",
  "retirementType": "END_OF_SALE",
  "effectiveAt": "2026-09-01T00:00:00+03:00",
  "existingSubscriberPolicy": "KEEP_AS_IS",
  "reasonCode": "PRODUCT_REPLACED_BY_100M"
}
```

Flow:

1. Validate package exists.
2. Check current active subscriptions through subscription read API if END_OF_LIFE or RETIRE_VERSION.
3. If active subscriptions exist and policy is MIGRATE_REQUIRED, require migrationWorkflowRef.
4. Run package-retirement-policy.drl.
5. Evaluate approval through EM-CFG-04 when required.
6. Schedule retirement.
7. Worker executes at effectiveAt.

Execution behavior:

- END_OF_SALE: mark availability rows ENDED; leave package/version valid for existing subscriptions.
- END_OF_LIFE: allowed only if no blocking active dependencies or approved migration plan exists.
- RETIRE_VERSION: supersede/archive one version; do not break subscriptions that reference it historically.

13. Step 12 - Cache Invalidation

Use Redis keys like:

```
catalog:package-availability:{operatorCode}:{channelCode}:{franchiseId}:{techRegionCode}:{date}
```

Invalidate when these events happen:

- PackageActivated
- PackageAvailabilityChanged
- PackageEndOfSale
- PackageRetired
- SIP-01 package/version updated
- PLM-CFG-01 service status changed

Do not rely only on TTL. Use event invalidation plus a short TTL.

14. Step 13 - Events

Use outbox rows created in the same transaction as lifecycle state changes.

Example:

```
{
  "eventType": "PackageActivated",
```



```

    "eventId": "evt-plp-001-activated",
    "occurredAt": "2026-07-01T00:00:05+03:00",
    "operatorCode": "WIK",
    "packageId": "pkg_01HX_100M",
    "packageCode": "FIBER_100M_RES",
    "packageVersionId": "pkv-100m-202607",
    "launchPlanId": "plp-001",
    "availability": [
      {
        "franchiseId": "fr-019",
        "techRegionCode": "NRB-WEST",
        "channelCode": "SALES_APP"
      }
    ]
  }
}

```

Consumers:

- FUL-02 refreshes sellable package cache.
- sales app refreshes offline catalog sync.
- Backoffice refreshes package status.
- REP records commercial launch KPI.
- SIP-04 validates bundle components against active packages.

15. Step 14 - Error Handling

Error	HTTP	Meaning
PACKAGE_NOT_FOUND	404	SIP-01 package does not exist.
PACKAGE_VERSION_NOT_FOUND	404	SIP-01 package version does not exist.
VERSION_NOT_IN_PACKAGE	400	Version belongs to another package.
VALIDATION_FAILED	422	One or more launch checks failed.
APPROVAL_REQUIRED	202	EM-CFG-04 approval request started.
APPROVAL_REJECTED	409	Launch cannot continue after rejection.
LAUNCH_NOT_DUE	409	Activation requested before launch date.
PACKAGE_ALREADY_ACTIVE	409	Duplicate activation attempt.
AVAILABILITY_NOT_FOUND	404	No matching availability row.
RETIREMENT_BLOCKED_ACTIVE_SUBSCRIPTIONS	409	Retirement would break active customers.

16. Step 15 - Frontend Flow

Backoffice implementation should call modules in this order:

1. Create/edit package definition through SIP-01.
2. Create launch plan through SIP-02.
3. Validate launch plan through SIP-02.
4. Display launch checks with PASS/WARN/FAIL.
5. Submit for review through SIP-02.
6. If approval required, show EM-CFG-04 approval request/task link.
7. After approval, show scheduled activation state.
8. Once active, sales app/order capture uses GET /api/packages/available.

The frontend should not call SIP-01 and directly set package status to active. Activation must go through SIP-02.

17. Worker Jobs

Implement as scheduled commands/jobs in the catalog worker deployment:

Job	Command name suggestion	What it does
Launch activation	package:launch:activate-due	Activates approved due launch plans.
Version cutover	package:version:cutover-due	Completes scheduled version cutovers.
Retirement	package:retirement:run-due	Applies end-of-sale/end-of-life plans.
Outbox publisher	package:lifecycle:outbox-publish	Publishes package lifecycle events.
Cache refresh	package:availability:refresh-cache	Warms or invalidates availability cache.

One worker process can host all these command handlers. Separate metrics by job name.

18. Tests To Write First

1. Create launch plan for existing package/version.
2. Create launch plan for missing version returns 404.
3. Validation fails when service is inactive.
4. Validation passes with warning when Drools requires approval.
5. Submit review calls EM-CFG-04 and stores approval request.
6. Approval callback approved moves plan to scheduled/approved.
7. Approval callback rejected moves plan to rejected.
8. Activation worker updates SIP-01 current version and availability.
9. Available packages API filters by channel/franchise/region/date.
10. Suspension removes package from available API for that scope.
11. End-of-sale blocks new sales but does not alter existing subscriptions.
12. Retirement with active subscriptions and no migration plan is blocked.

19. Developer Checklist

- SIP-02 tables and indexes created.
- Launch plan create API implemented.
- Validation API implemented with SIP-01, PLM-CFG, BIL-CFG calls.
- Drools launch facts and rules integrated.
- Submit-review flow calls EM-CFG-04.
- Approval callback is idempotent.
- Activation worker updates package/version state through SIP-01 service boundary.
- Availability API implemented and cached.
- Suspend/resume availability implemented.
- Retirement plan flow implemented.
- Outbox events implemented.
- Backoffice flow wired to SIP-01, SIP-02, and EM-CFG-04 APIs.
- Tests cover launch, approval, activation, availability, and retirement.